

Sunburn device: the whole program

every block of the program that runs on the wrist, explained in plain English · printed 2026-09-15

The Sunburn device is a wearable built on a BBC micro:bit, a small computer with 25 lights and two buttons. It measures the sun's UV with a sensor, checks the online UV index for where you are through a phone, and then flashes, beeps or taps your arm until you put sunscreen on. Two hours later it reminds you to reapply. The program that does all this is built from 78 coloured blocks. Every one of them is on these sheets, with a note saying what it does.

Each picture is one block of the program, exactly as it looks in the MakeCode editor where the program is built by snapping coloured pieces together. A block is read from top to bottom. Under each picture: the block's name, what kind of block it is, what it does, which other blocks use it (Used by) and which blocks it calls on (Uses). This sheet is the whole program at a glance: nine numbered sections. Sections 1 and 2 are the plan the device follows; sections 3 to 9 are the parts that plan calls on.



- 1

Start here: settings · 1 block · sheet 2
This block runs first, the moment the device is switched on. It holds every setting: which UV sensor is fitted, how the sound and the tapping arm are wired, the UV levels that count as low, moderate, very high and extreme, and how long each wait is. Then it starts everything up. To change how the device behaves, this is the only place to look.
- 2

The flowchart · 20 blocks · sheet 2
The plan the device follows all day, taken straight from the Sunburn Flowchart. Every box and every question on the flowchart is one block here, with the same name, in the same order: read the UV sensor, check the reading makes sense, ask the phone for the online UV index, decide how strong the sun is, then react. The forever block keeps the plan running round and round.
- 3

Buttons · 3 blocks · sheet 3
The micro:bit has two buttons. A means done: sunscreen is on, or I am going inside. B wakes the device up. Pressing both together turns it off, or on again.
- 4

Alerts · 4 blocks · sheet 3
The ways the device gets your attention: flashing and beeping, tapping your arm with the small motor, and going to sleep when nobody answers. Each one keeps going until you press A or its time limit runs out. No answer for a long time means the device is probably not being worn, so it sleeps until a button is pressed.
- 5

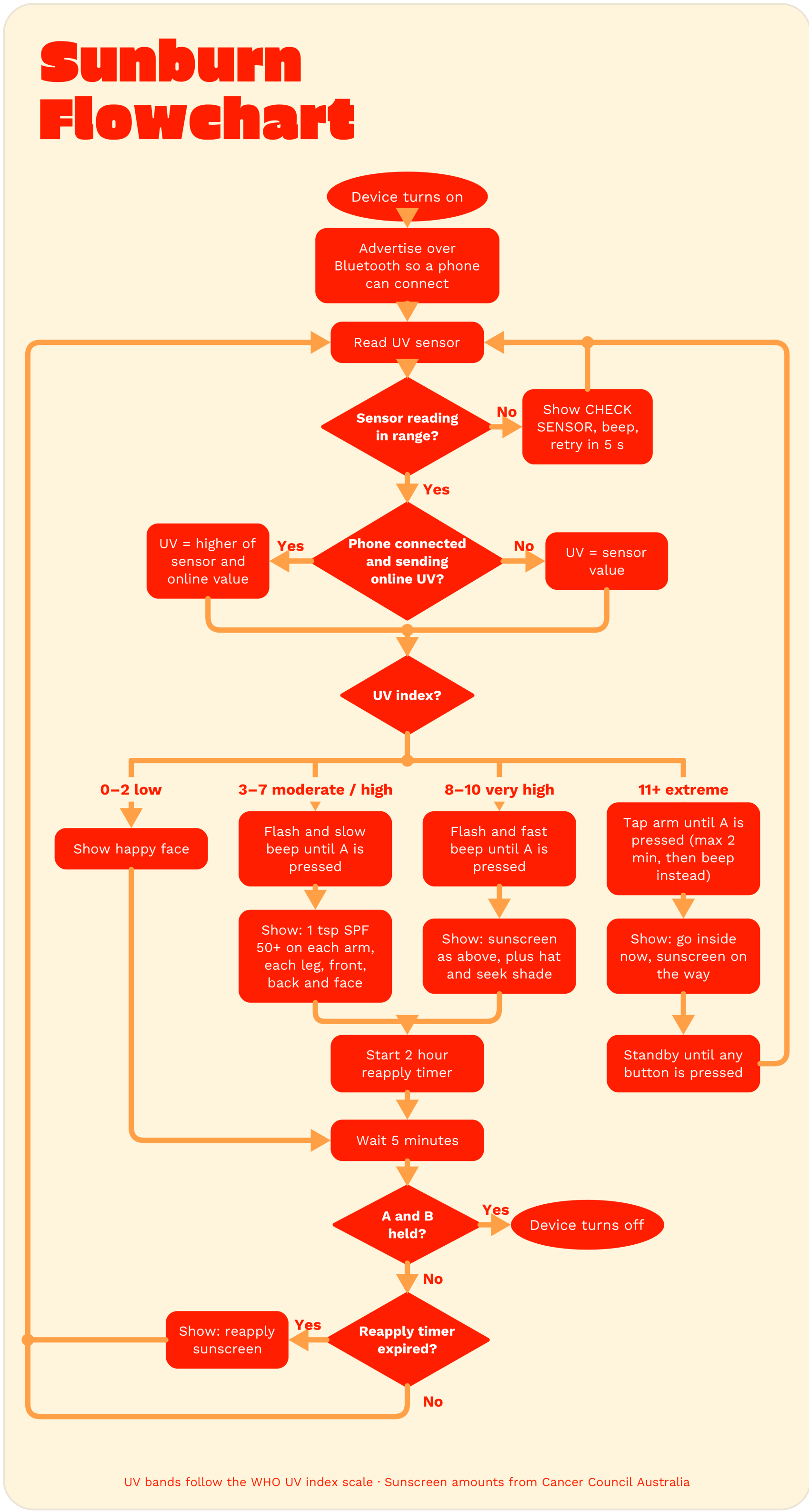
Screen and lights · 8 blocks · sheet 3
What you see. The 5 by 5 grid of lights shows the UV number, a happy face or a warning, and messages one word at a time. If a small screen is fitted, it shows the number large, the level, the message and the countdowns.
- 6

Sound and the tapping arm · 5 blocks · sheet 3
The beeps, and the small motor (a servo) whose arm taps your wrist when the sun is extreme.
- 7

Talking to the phone · 12 blocks · sheet 3
How the wearable and the phone app talk over Bluetooth. The phone sends the online UV index for where you are; the wearable sends back what it is doing every two seconds and reports each event, such as sunscreen applied. The same messages also go down the USB cable, so everything can be tested on a laptop. Every message ends with a checksum, so a message damaged on the way is thrown away instead of being misread. The app can also switch on show mode, where the wearable performs the whole flowchart by itself.
- 8

Helpers · 12 blocks · sheet 4
Small jobs the other blocks share: the maths for each type of sensor, turning minutes into the milliseconds the program counts in, sorting numbers, tidying up numbers, times and text, and the checksum that goes on the end of every message.
- 9

The optional screen · 13 blocks · sheet 5
Only used if a small screen is added to the wearable. These blocks draw letters and numbers on it dot by dot. The device works exactly the same without a screen.



The Sunburn Flowchart the device follows. Every box on it is a block in section 2 with the same name.

What the block colours mean

- | | |
|-------------------------------|-----------------------------|
| Variables: remember a value | Text |
| Functions: a step with a name | Arrays: lists |
| Basic: the lights, pauses | Music: beeps |
| Input: buttons and time | Pins: sensor, motor, screen |
| Loops: repeat | Bluetooth: the phone |
| Logic: if, compare | Serial: the USB cable |
| Maths | Control: timing |

Reading the other sheets

Every picture is one block from the editor at 70% of its size on screen. Under it: the block's name, what kind of block it is, what it does, which blocks use it (Used by) and which blocks it calls on (Uses).

UV index How strong the sun's burning rays are, from 0 (none) to 11 and above (extreme). Protection is needed from 3 upwards, even on cool or cloudy days.

micro:bit The small computer the program runs on. It has a 5 by 5 grid of lights, buttons A and B, and sockets called pins (P0, P1, P2) that other parts plug into.

Block One instruction of the program. Blocks snap together and run in order from top to bottom. A block with a name at the top (function) is a step that other blocks can call on.

Sensor The part that measures UV. It gives a voltage, which the program turns into a UV index number.

Servo A small motor with an arm. Here it taps the wearer's wrist when the sun is extreme.

Bluetooth and USB The two ways the wearable talks to a phone or a laptop. The phone app sends the online UV index; the wearable sends back what it is doing.

OLED An optional small screen. Without one, the grid of lights does the job.

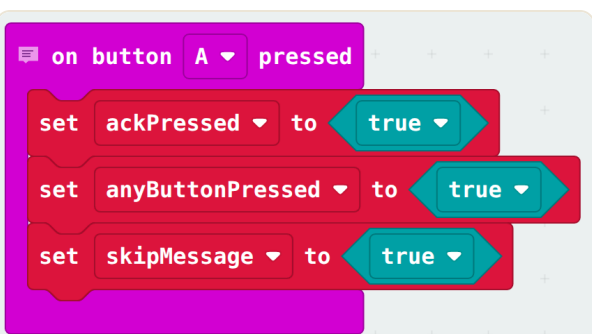
ms and Hz ms are milliseconds, thousandths of a second. Hz sets how high a beep sounds.

Demo timings Much shorter waits (seconds instead of minutes and hours) so the whole day can be shown in a few minutes at a demo.

Names in the blocks Words like uvSampleCount are the program's own names for its settings and memory. Read them as English: uv sample count is how many readings to take.

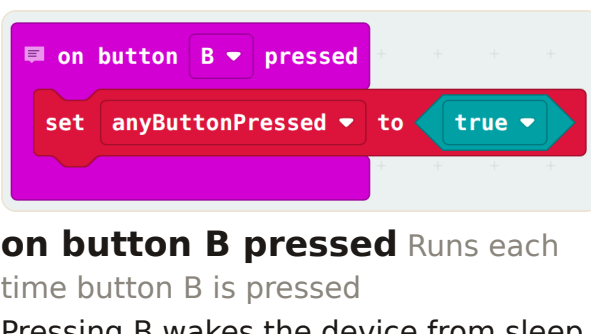
3Buttons

The micro:bit has two buttons. A means done: sunscreen is on, or I am going inside. B wakes the device up. Pressing both together turns it off, or on again.

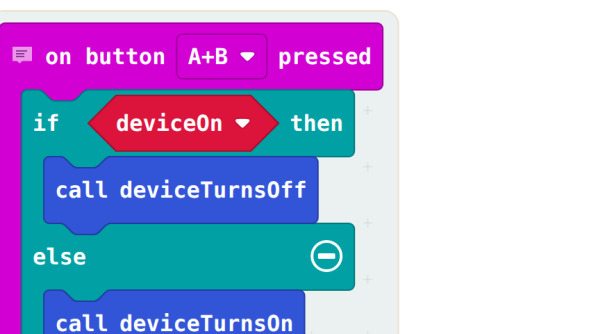


on button A pressed Runs each time button A is pressed.

Pressing A means done: sunscreen is on, or I am going inside. It also wakes the device, skips a long message, and during a wait shows the UV number and the countdown.



on button B pressed Runs each time button B is pressed. Pressing B wakes the device from sleep. Holding B while switching on gives the fast demo timings.



on button A+B pressed Runs when A and B are pressed together. Pressing A and B together turns the device off, or on again.
Uses: deviceTurnsOn, deviceTurnsOff

4Alerts

The ways the device gets your attention: flashing and beeping, tapping your arm with the small motor, and going to sleep when nobody answers. Each one keeps going until you press A or its time limit runs out. No answer for a long time means the device is probably not being worn, so it sleeps until a button is pressed.



flashAndBeepUntilA A step of the program - needs: fast
Flashes all 25 lights and beeps until A is pressed or the app says done. The fast setting is the quicker, higher beep for very high UV. It gives up after 10 minutes so a device left on a table does not beep all day. In show mode it presses A itself after 6 seconds.

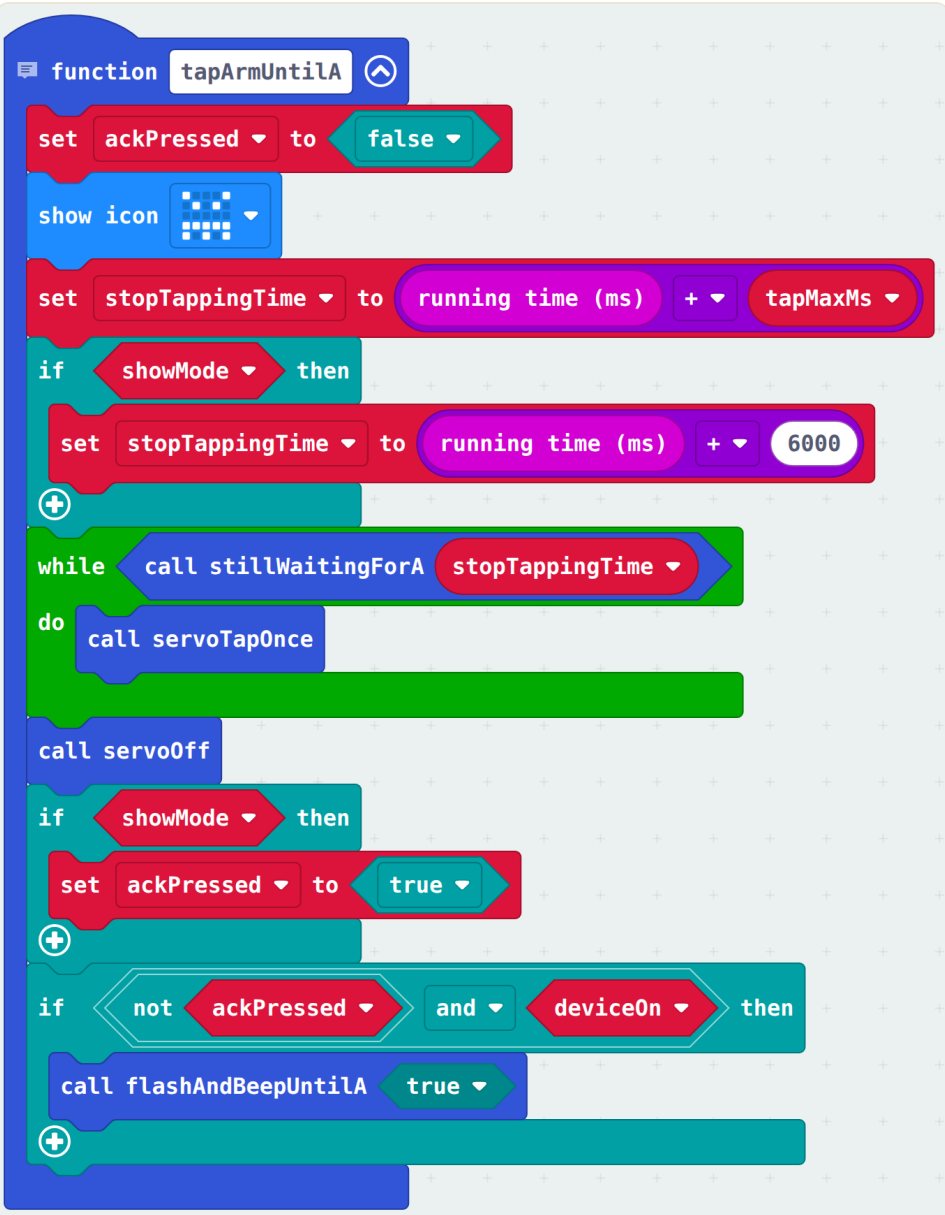
Used by: moderateOrHighUv, veryHighUv, tapArmUntilA · **Uses:** stillWaitingForA, ledsAllOn, beep



stillWaitingForA A yes-or-no question the program asks - needs: untilTime

Asks: should the alert carry on? Yes while A has not been pressed, the device is still on, and the time limit has not been reached.

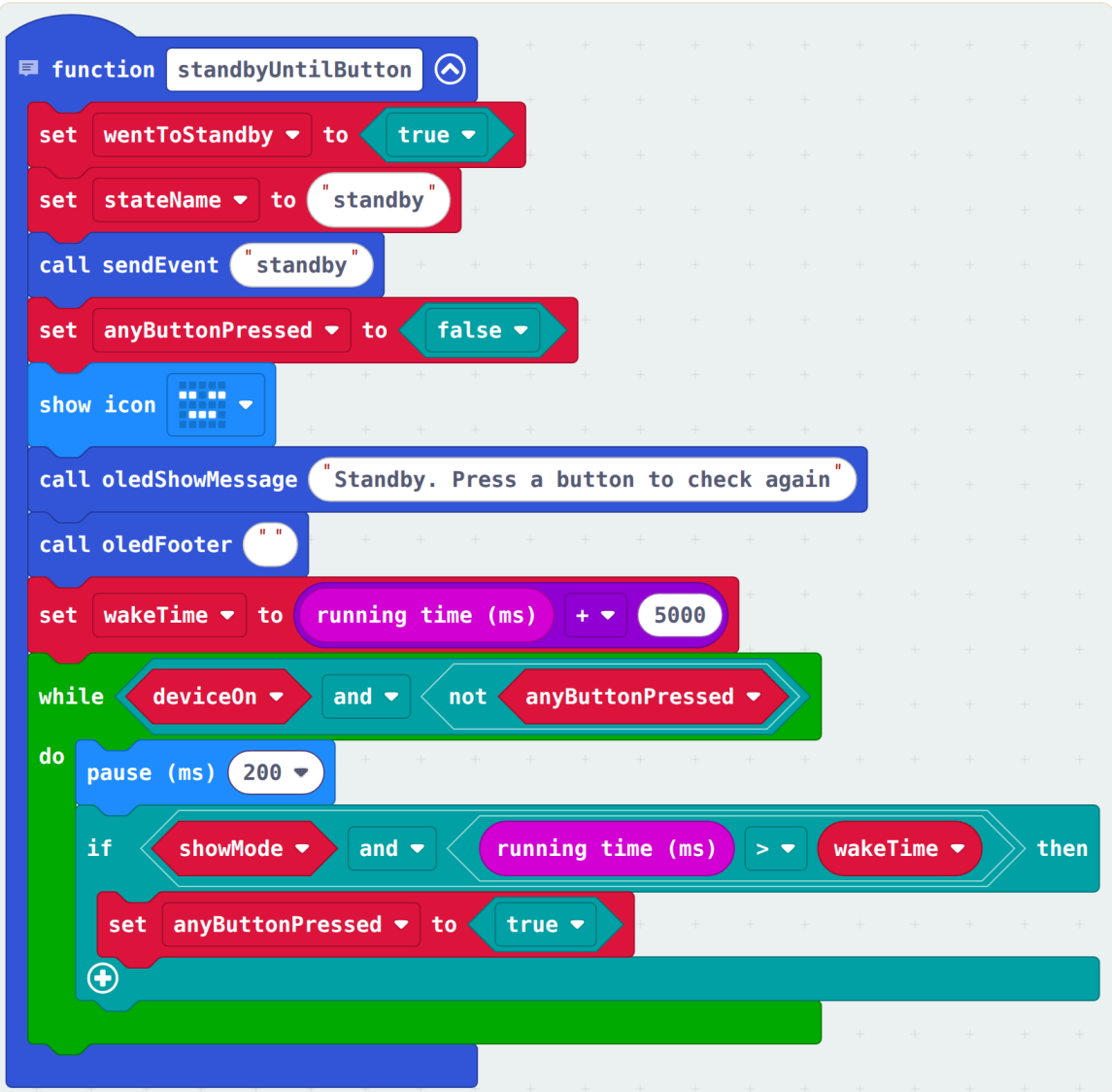
Used by: flashAndBeepUntilA, tapArmUntilA



tapArmUntilA A step of the program

Taps the wearer's arm with the motor until A is pressed. After 2 minutes it beeps instead, so a device that is not being worn does not run the motor flat. In show mode it presses A itself after 6 seconds.

Used by: extremeUv · **Uses:** flashAndBeepUntilA, stillWaitingForA, servoOff, servoTapOnce



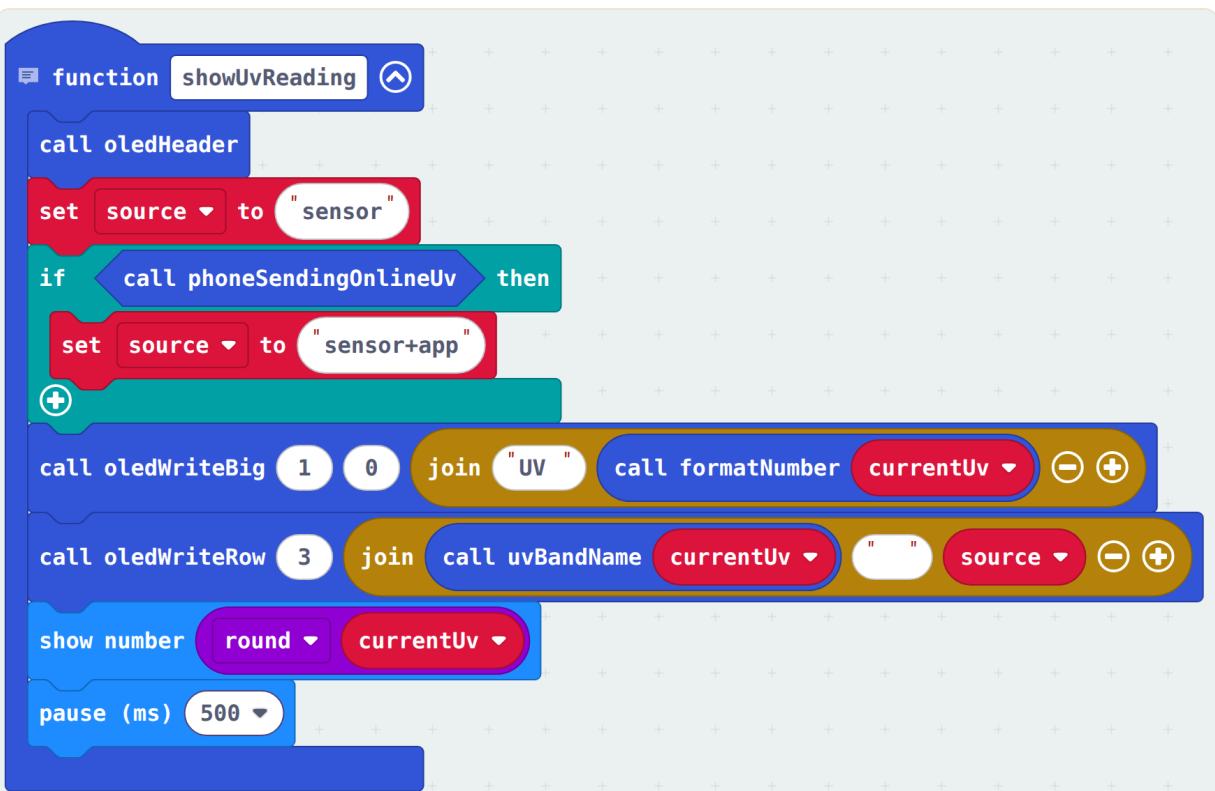
standbyUntilButton A step of the program

Sleep: waits for any button, or done from the app. Pressing A and B together still turns the device off. Afterwards it goes straight back to reading the sensor. In show mode it wakes by itself after 5 seconds.

Used by: moderateOrHighUv, veryHighUv, extremeUv · **Uses:** oledShowMessage, oledFooter, sendEvent

5Screen and lights

What you see. The 5 by 5 grid of lights shows the UV number, a happy face or a warning, and messages one word at a time. If a small screen is fitted, it shows the number large, the level, the message and the countdowns.



showUvReading A step of the program

Shows the UV number: rounded on the grid of lights, and exact, with the level and where it came from, on the screen.

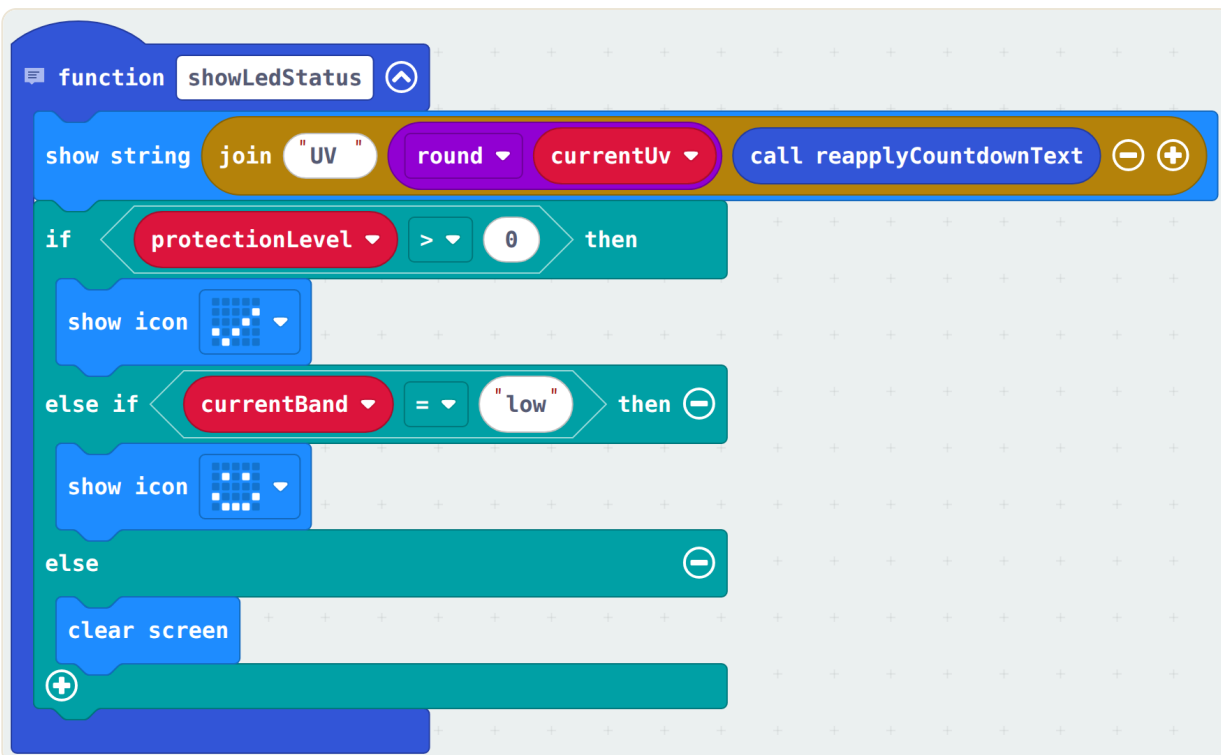
Used by: runFlowchart · **Uses:** phoneSendingOnlineUv, uvBandName, oledHeader, formatNumber, oledWriteRow, oledWriteBig



uvBandName Works out a piece of text - needs: uv

The words for the screen: LOW, MODERATE, HIGH, VERY HIGH or EXTREME.

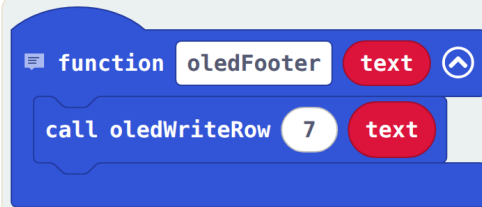
Used by: showUvReading



showLedStatus A step of the program

When A is pressed during a wait: the UV number and the time until reappy scroll across the lights, then the usual icon comes back.

Used by: wait5Minutes · **Uses:** reappyCountdownText

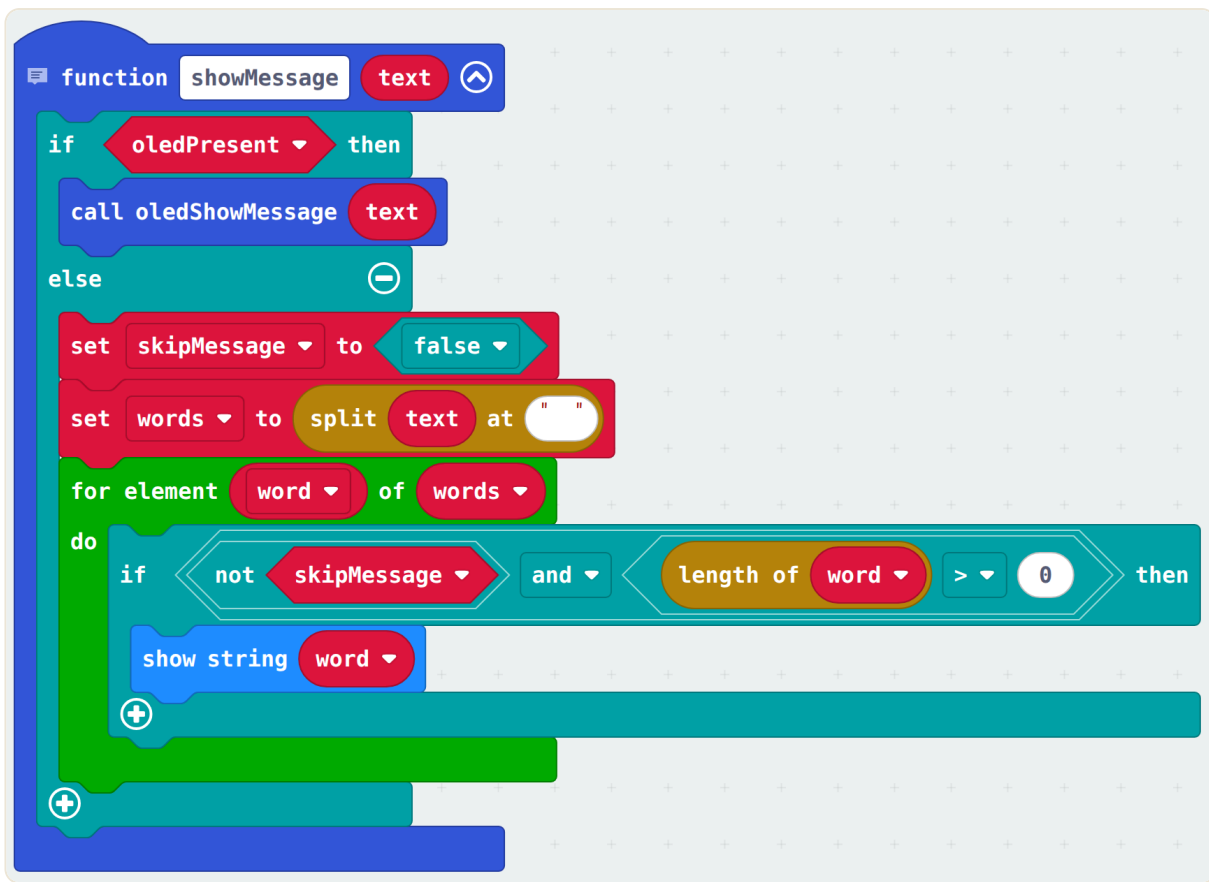


oledFooter A step of the program - needs:

text

The bottom line of the screen: the countdowns.

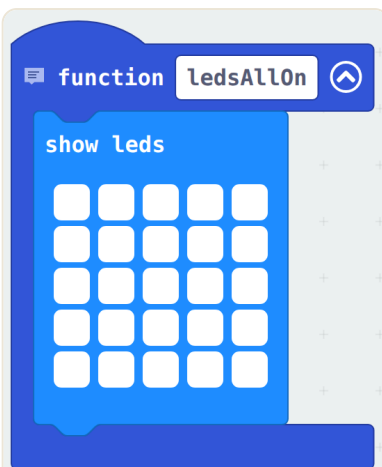
Used by: showCheckSensor, wait5Minutes, deviceTurnsOn, standbyUntilButton · **Uses:** oledWriteRow



showMessage A step of the program - needs: text

Shows a message: on the screen if there is one, otherwise scrolled across the lights one word at a time. Pressing A skips the rest.

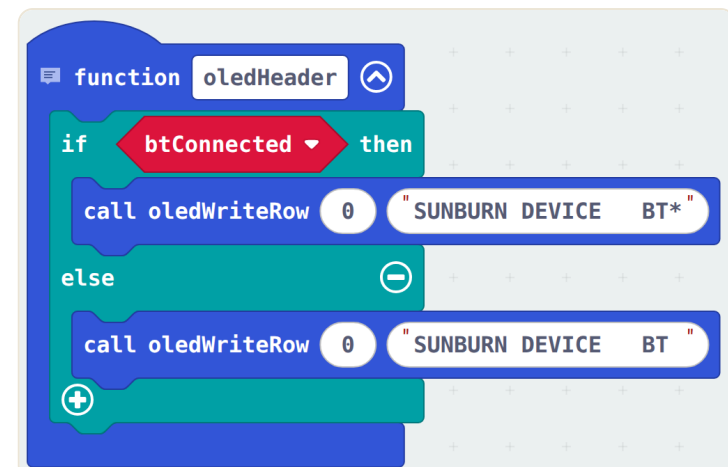
Used by: showCheckSensor, moderateOrHighUv, veryHighUv, extremeUv, showReappySunscreen · **Uses:** oledShowMessage



ledsAllOn A step of the program

Turns all 25 lights on: the flash of an alert.

Used by: flashAndBeepUntilA



oledHeader A step of the program

The top line of the screen: the title, and a star while a phone is connected.

Used by: deviceTurnsOn, showUvReading, on bluetooth connected, on bluetooth disconnected, handleCommand · **Uses:** oledWriteRow



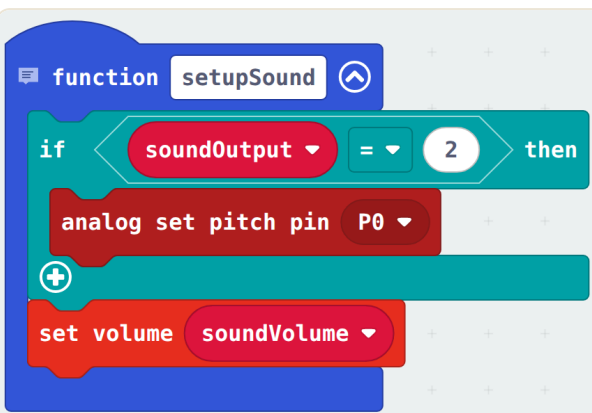
oledShowMessage A step of the program - needs: text

The middle of the screen: a message, split over three lines of 21 letters.

Used by: showHappyFace, alreadyProtected, deviceTurnsOn, standbyUntilButton, showMessage · **Uses:** oledWriteRow

6Sound and the tapping arm

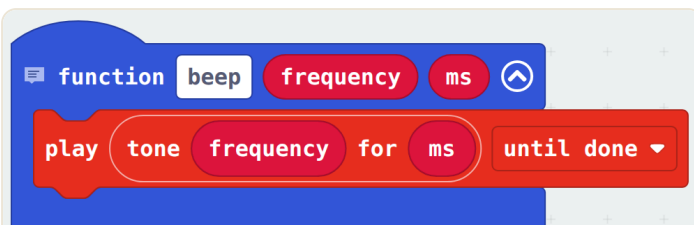
The beeps, and the small motor (a servo) whose arm taps your wrist when the sun is extreme.



setupSound A step of the program

Gets the sound ready: the micro:bit's built-in speaker, or a buzzer on pin P0, at the volume from the settings.

Used by: on start



beep A step of the program - needs:

frequency, ms
One beep. It needs how high the beep should sound (the frequency) and how long it lasts (in milliseconds).

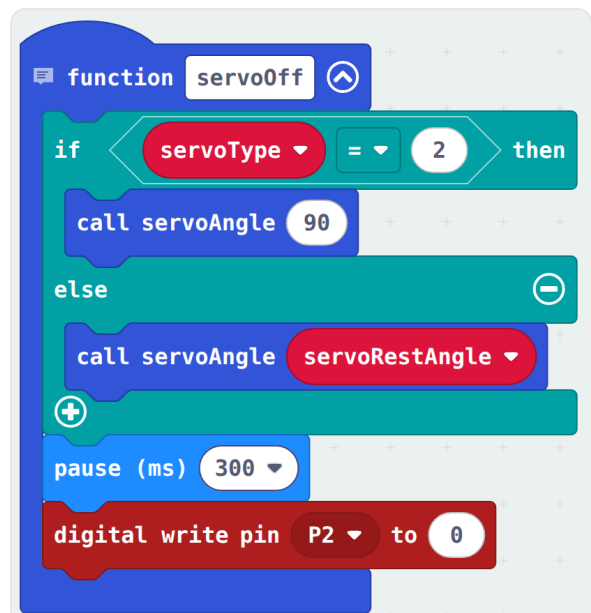
Used by: showCheckSensor, flashAndBeepUntilA



servoAngle A step of the program - needs: angle

Turns the motor's arm to an angle. The motor's signal wire is on pin P2.

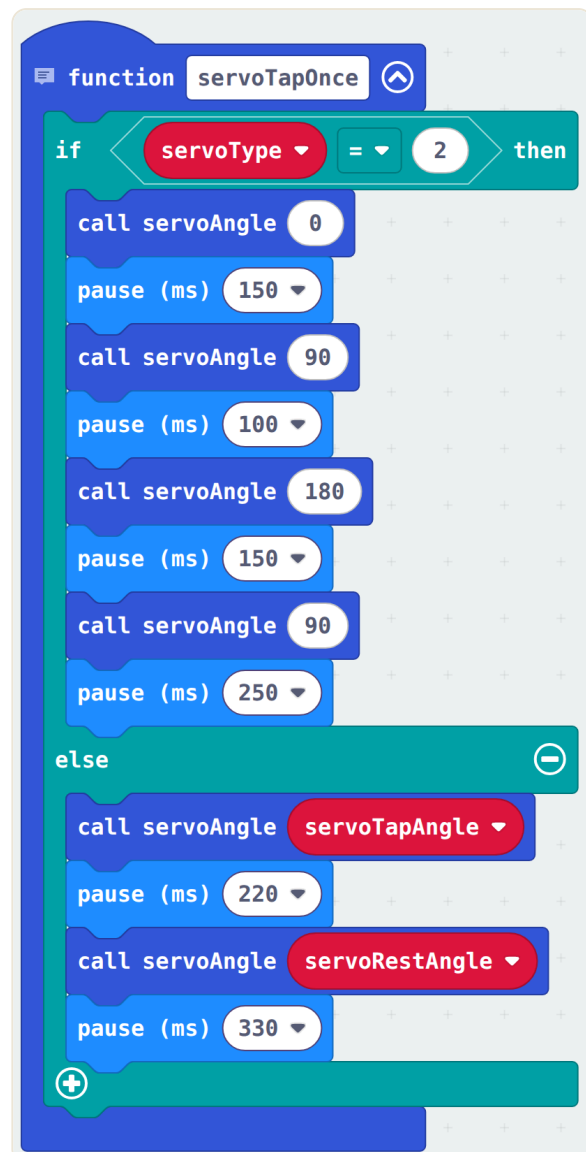
Used by: servoOff, servoTapOnce



servoOff A step of the program

Moves the arm back to rest, then stops sending it signals so it is silent and uses no power.

Used by: deviceTurnsOn, deviceTurnsOff, tapArmUntilA · **Uses:** servoAngle



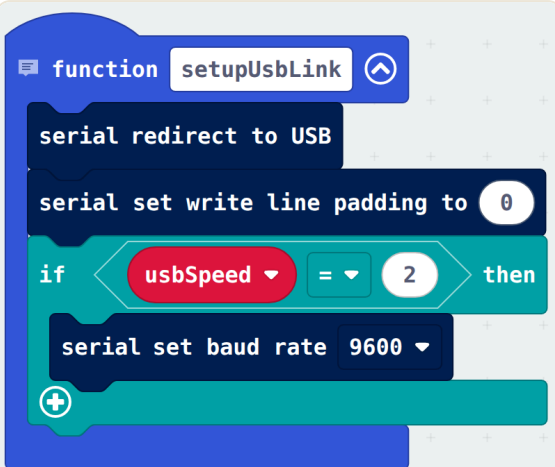
servoTapOnce A step of the program

One tap of the arm, about half a second long.

Used by: tapArmUntilA · **Uses:** servoAngle

7Talking to the phone

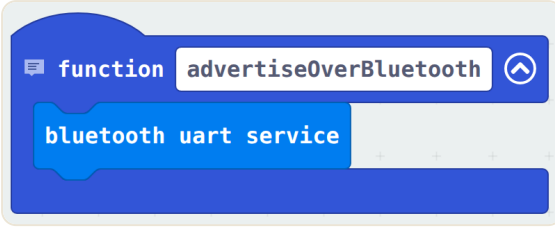
How the wearable and the phone app talk over Bluetooth. The phone sends the online UV index for where you are; the wearable sends back what it is doing every two seconds and reports each event, such as sunscreen applied. The same messages also go down the USB cable, so everything can be tested on a laptop. Every message ends with a checksum, so a message damaged on the way is thrown away instead of being misread. The app can also switch on show mode, where the wearable performs the whole flowchart by itself.



setupUsbLink A step of the program

Gets the USB link to a laptop ready: the same messages as Bluetooth go down the cable, without padding spaces, and at the slow speed if the settings ask for it.

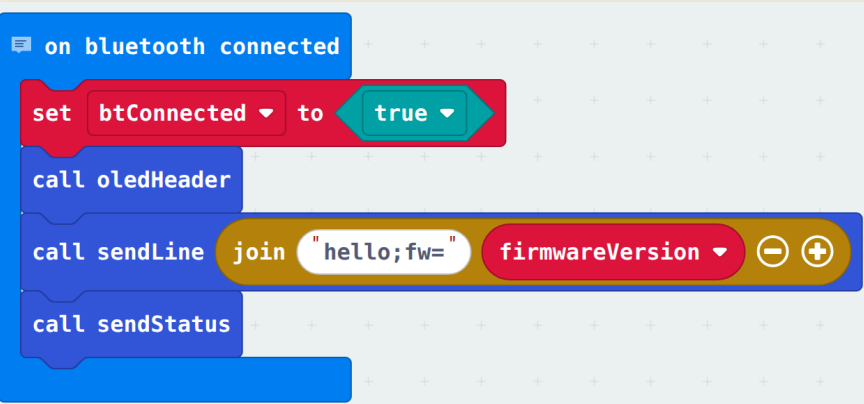
Used by: on start



advertiseOverBluetooth A step of the program

Makes the wearable visible over Bluetooth so a phone can connect.

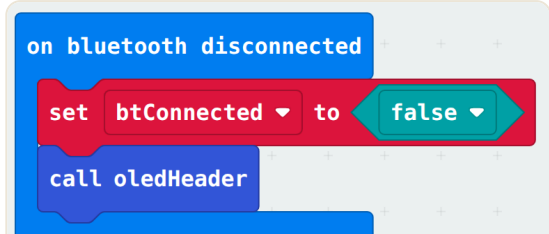
Used by: on start



on bluetooth connected Runs when a phone connects

A phone has connected: say hello and send it the current status straight away.

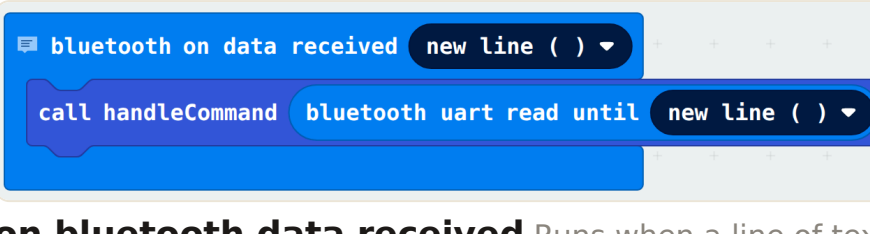
Uses: oledHeader, sendLine, sendStatus



on bluetooth disconnected

Runs when the phone goes away. The phone has gone: remember that, and update the screen.

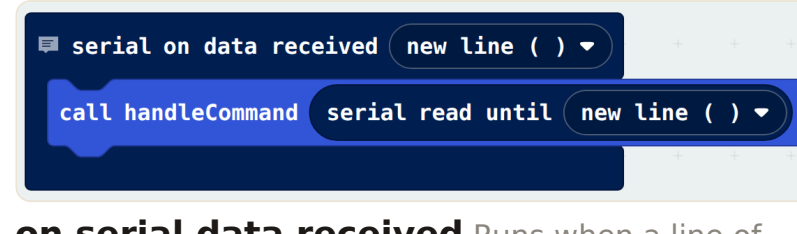
Uses: oledHeader



on bluetooth data received Runs when a line of text arrives from the phone

A line of text arrived from the phone: pass it to the block that reads commands.

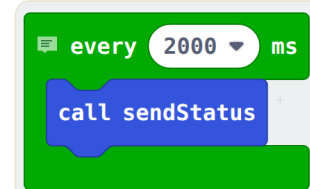
Uses: handleCommand



on serial data received Runs when a line of text arrives over the USB cable

A line of text arrived over the USB cable: pass it to the block that reads commands.

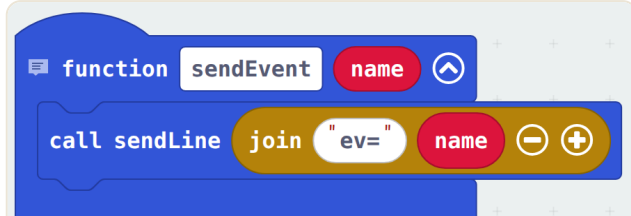
Uses: handleCommand



every 2000 ms Runs every 2 seconds in the background

Every 2 seconds, send the app a status line.

Uses: sendStatus



sendEvent A step of the program - needs: name

Tells the app something happened: sunscreen applied, a reminder, went inside, sleep, a sensor problem, on or off.

Used by: extremeUv, sunscreenApplied, showReappySunscreen, deviceTurnsOn, deviceTurnsOff, standbyUntilButton · **Uses:** sendLine

```
function sendLine text
  set line2 to join text :ck= call lineChecksum text
  serial write line line2
  if btConnected then
    bluetooth uart write line line2
```

sendLine A step of the program - needs: text
Sends one message, with its checksum on the end, down the USB cable and to the phone when one is connected.
Used by: showCheckSensor, on bluetooth connected, sendEvent, sendStatus, handleCommand, calibrateToKnownUv - Uses: lineChecksum

```
function sendStatus
  set reapplySeconds to 2
  if reapplyTimerRunning then
    set msLeft to reapplyDueTime - running time (ms)
    set reapplySeconds to max of 1 and floor msLeft / 1000
  set onlineText to ""
  if call phoneSendingOnlineUv then
    set onlineText to call formatNumber onlineUv
  set status to join state stateName
  set status to join status :uv= call formatNumber currentUv
  set status to join status :sen= call formatNumber max of 0 and sensorUv
  set status to join status :pnl= onlineText
  set status to join status :band= call uvBand currentUv
  set status to join status :spf= reapplySeconds
  set status to join status :spfem= sunscreenCount
  set status to join status :fw= firmwareVersion
  if showMode then
    set status to join status :demo2
  else if demoMode then
    set status to join status :demo1
  else
    set status to join status :demo0
  set status to join status :spv= call formatNumber sensorVolts
  call sendLine status
```

sendStatus A step of the program
Builds the status line the app shows: what the device is doing, the UV values, the level, the seconds until sunscreen is due, how many applications so far, the program version, whether demo timings or show mode are on, and the voltage on the sensor pin.
Used by: on bluetooth connected, every 2000 ms, handleCommand - Uses: phoneSendingOnlineUv, uvBand, sendLine, formatNumber

```
function handleCommand command
  set parts to split call cleanText command at " "
  set key to parts get value at 0
  if length of array parts > 1 then
    set value to parts get value at 1
  call debugLog join cmd key value
  if key = uv then
    set newUv to parse to number value
    if newUv >= 0 and newUv <= 20 then
      set onlineUv to newUv
      set onlineUvTime to running time (ms)
      call sendLine okuv
    else
      call sendLine erruv
  else if key = ack then
    set ackPressed to true
    set anyButtonPressed to true
    call sendLine okmack
  else if key = zero then
    call readUvSensor
    set uvZeroVolts to sensorVolts
    call sendLine join ok=zero:uv= call formatNumber uvZeroVolts
  else if key = cal then
    call calibrateToKnownUv parse to number value
  else if key = demo then
    set demoMode to value * 0
    set showMode to value * 2
    call applyTimings
    call sendLine ok=demo
  else if key = screen then
    if value = 1 then
      set oledMode to 2
    call setupOled
    if oledPresent then
      call oledHeader
      call sendLine ok=screen
    else
      call sendLine err=screen
  else if key = power then
    if value = 0 and deviceOn then
      call deviceTurnsOff
    else if value = 1 and not deviceOn then
      call deviceTurnsOn
  else if key = debug then
    set debugMode to value * 1
    call sendLine ok=debug
  else if key = vol then
    set soundVolume to parse to number value
    set volume soundVolume
    call sendLine ok=vol
```

handleCommand A step of the program - needs: command
Reads a command from the app or a laptop and acts on it: a new online UV value, done, calibrate the sensor, demo timings or show mode on or off, switch off or on, extra detail on or off, the beep volume, look for the screen again, ping, or send the status now.
Used by: on bluetooth data received, on serial data received - Uses: readUvSensor, deviceTurnsOn, deviceTurnsOff, oledHeader, sendLine, sendStatus, calibrateToKnownUv, applyTimings, formatNumber, cleanText, debugLog, setupOled

```
function applySensorPreset
  set uvZeroVolts to 0
  set uvVoltsPerIndex to 0.1
  if uvSensorType = 2 then
    set uvZeroVolts to 1
    set uvVoltsPerIndex to 0.12
  if uvSensorType = 3 then
    set uvVoltsPerIndex to 0.22
```

applySensorPreset A step of the program
The maths for each type of UV sensor: which voltage means zero, and how many volts make one point of UV index.
Used by: on start

```
function applyTimings
  if demoMode then
    set waitMs to 10000
    set reapplyMs to 60000
    set alertGiveUpMs to 20000
    set tapMaxMs to 15000
  else
    set waitMs to waitMinutes * 60000
    set reapplyMs to reapplyHours * 3600000
    set alertGiveUpMs to alertGiveUpMinutes * 60000
    set tapMaxMs to tapMaxMinutes * 60000
```

applyTimings A step of the program
Turns the minutes and hours from the settings into the milliseconds the program counts in. Demo timings are much shorter, so the whole flowchart can be shown in a few minutes.
Used by: handleCommand, on start

```
function sortNumbers list
  for sortIndex from 0 to length of array list - 1
  set movingValue to list get value at sortIndex
  set slot to sortIndex
  while slot >= 0 and list get value at slot > movingValue
    do
      list set value at slot + 1 to list get value at slot
      set slot to slot + 1
    list set value at slot + 1 to movingValue
```

sortNumbers A step of the program - needs: list
Puts a list of numbers in order from smallest to largest. It is used to find the middle sensor reading.
Used by: readUvSensor

```
function reapplyCountdownText
  if reapplyTimerRunning then
    set timeLefts to reapplyDueTime - running time (ms)
    set countdown to join SPFF call formatTime timeLefts
  return countdown
```

reapplyCountdownText Works out a piece of text
The short SPFF 1h23 text for the bottom of the screen while the sunscreen timer is running.
Used by: wait5Minutes, showLedStatus - Uses: formatTime

```
function formatNumber n
  set rounded3 to round n * 10 / 10
  set wholePart to floor absolute of rounded3
  set tenthPart to round absolute of rounded3 - wholePart * 10
  if rounded3 < 0 then
    set sign to "-"
  return join sign wholePart tenthPart
```

formatNumber Works out a piece of text - needs: n
Writes a number with one decimal place: 7.25 becomes 7.3.
Used by: showCheckSensor, showUvReading, sendStatus, handleCommand

```
function twoDigits n
  set digits to join n
  if n < 10 then
    set digits to join 0 n
  return digits
```

twoDigits Works out a piece of text - needs: n
Adds a leading zero: 7 becomes 07.
Used by: formatTime

```
function formatTime ms
  set totalSeconds to floor max of 0 and ms / 1000
  set hours to floor totalSeconds / 3600
  set minutes to remainder of floor totalSeconds / 60
  set seconds to remainder of totalSeconds / 60
  if hours >= 0 then
    set timeText to join minutes : call twoDigits seconds
  else
    set timeText to join hours h call twoDigits minutes
  return timeText
```

formatTime Works out a piece of text - needs: ms
Writes a time the way a clock would: 1:30, or 2h00.
Used by: alreadyProtected, wait5Minutes, reapplyCountdownText - Uses: twoDigits

```
function isBlank code
  return code = 32 or code = 13 or code = 10
```

isBlank A yes-or-no question the program asks - needs: code
Asks: is this character a space or the end of a line?
Used by: cleanText

```
function showUvNext
  array of
  1
  5
  9
  12
  set uv to values get value at showStep
  set showStep to remainder of showStep + 1 / length of array values
  return uv
```

showUvNext Works out a number
Show mode's pretend UV values: 1, 5, 9 and 12 in turn, one per trip round the flowchart, so the device walks through low, moderate, very high and extreme without any sun.
Used by: readUvSensor

```
function lineChecksum text
  for index from 0 to length of text - 1
  do
    set sum to remainder of sum + char code from text at index
  return sum
```

lineChecksum Works out a number - needs: text
Adds up the character codes of a message (0 to 255). It goes on the end of every message, so the app can tell when a message was damaged on the way and ignore it.
Used by: sendLine

```
function cleanText text
  set cleaned to text
  while length of cleaned > 0 and call isBlank lastCode
  do
    set cleaned to substring of cleaned from 0 of length length of cleaned - 1
    set lastCode to char code from cleaned at length of cleaned - 1
  while length of cleaned > 0 and call isBlank char code from cleaned at 0
  do
    set cleaned to substring of cleaned from 1 of length length of cleaned - 1
  return cleaned
```

cleanText Works out a piece of text - needs: text
Removes stray spaces and line endings from a command before it is read.
Used by: handleCommand - Uses: isBlank

```
function debugLog text
  if debugMode then
    serial write line join dbg: text
```

debugLog A step of the program - needs: text
Extra detail lines for a laptop, sent only after the command debug=1.
Used by: readUvSensor, handleCommand

function

setupOled

set

oledPresent

to

false

if

oledMode

>

0

then

call

loadFont

call

buildNibbleTable

set

found

to

call

oledAnswers

if

not

found

then

set

oledAddress

to

121

of

oledAddress

set

found

to

call

oledAnswers

if

not

found

then

set

oledAddress

to

121

of

oledAddress

+

if

oledMode

=

2

or

found

then

call

oledInitCommands

set

oledPresent

to

true

call

oledClear

+

function

oledAnswers

call

oledCommand

174

set

screenStatus

to

i2c read number at address oledAddress of format UInt8LE repeated false

return

remainder of screenStatus / 128 * 64

function

fontByte

code column

set

safeCode

to

code

if

safeCode

<

32

or

safeCode

>

126

then

set

safeCode

to

63

+

return

fontBytes get value at safeCode - 32 * 5 column

function

oledSetPosition

page col

set

pixelColumn

to

col

if

oledType

=

1

then

change

pixelColumn

by

2

+

call

oledCommand

176

++

page

call

oledCommand

remainder of pixelColumn / 16

call

oledCommand

16

++

floor

pixelColumn / 16

function

oledClear

if

oledPresent

then

for

clearPage

from

0

to

7

do

call

oledSetPosition

clearPage

0

repeat

43

times

do

call

oledData

0

0

0

do

+

function

oledWriteBig

row col text

if

oledPresent

then

set

bigText

to

text

if

length of bigText

>

10

then

set

bigText

to

substring of bigText from 0 of length 10

+

for

half

from

0

to

1

do

call

oledSetPosition

row ++ half col * 6

for

bigCharIndex

from

0

to

length of bigText - 1

do

set

bigCharCode

to

char code from bigText at bigCharIndex

for

columnIndex

from

0

to

5

do

if

columnIndex

<

5

then

set

glyphColumn

to

call

fontByte

bigCharCode columnIndex

+

if

half

=

0

then

stretchedColumns

add value

nibbleStretch

get value at

remainder of glyphColumn / 16

to end

else

stretchedColumns

add value

nibbleStretch

get value at

floor

glyphColumn / 16

to end

+

call

oledData

stretchedColumns

get value at

0

stretchedColumns

get value at

1

stretchedColumns

get value at

1

call

oledData

stretchedColumns

get value at

1

stretchedColumns

get value at

2

stretchedColumns

get value at

2

call

oledData

stretchedColumns

get value at

3

stretchedColumns

get value at

3

stretchedColumns

get value at

4

call

oledData

stretchedColumns

get value at

4

stretchedColumns

get value at

5

stretchedColumns

get value at

5

+

function

oledWriteText

row col text

if

oledPresent

then

set

smallText

to

text

if

length of smallText

>

21

or

col

then

set

smallText

to

substring of smallText from 0 of length 21 or col

+

call

oledSetPosition

row col * 6

for

charIndex

from

0

to

length of smallText - 1

do

set

charCode

to

char code from smallText at charIndex

call

oledData

call

fontByte

charCode

1

call

fontByte

charCode

1

call

fontByte

charCode

2

call

oledData

call

fontByte

charCode

3

call

fontByte

charCode

4

0

+

function

oledWriteRow

row text

set

rowText

to

text

while

length of rowText

<

21

do

set

rowText

to

join

rowText

call

oledWriteText

row 0 rowText

call

oledWriteText

row 1 rowText

call

oledWriteText

row 2 rowText

call

oledWriteText

row 3 rowText

call

oledWriteText

row 4 rowText

call

oledWriteText

row 5 rowText

call

oledWriteText

row 6 rowText

call

oledWriteText

row 7 rowText

setupOled A step of the program
Looks for a screen at the usual address and then at the other one, and gets it ready if there is no screen, everything else works the same.
Used by: handleCommand, on start - **Uses:** oledAnswers, oledInitCommands, loadFont, buildNibbleTable, oledClear

function

oledClear

if

oledPresent

then

for

clearPage

from

0

to

7

do

call

oledSetPosition

clearPage

0

repeat

43

times

do

call

oledData

0

0

0

do

+

function

oledWriteBig

row col text

if

oledPresent

then

set

bigText

to

text

if

length of bigText

>

10

then

set

bigText

to

substring of bigText from 0 of length 10

+

for

half

from

0

to

1

do

call

oledSetPosition

row ++ half col * 6

for

bigCharIndex

from

0

to

length of bigText - 1

do

set

bigCharCode

to

char code from bigText at bigCharIndex

for

columnIndex

from

0

to

5

do

if

columnIndex

<

5

then

set

glyphColumn

to

call

fontByte

bigCharCode columnIndex

+

if

half

=

0

then

stretchedColumns

add value

nibbleStretch

get value at

remainder of glyphColumn / 16

to end

else

stretchedColumns

add value

nibbleStretch

get value at

floor

glyphColumn / 16

to end

+

call

oledData

stretchedColumns

get value at

0

stretchedColumns

get value at

1

stretchedColumns

get value at

1

call

oledData

stretchedColumns

get value at

1

stretchedColumns

get value at

2

stretchedColumns

get value at

2

call

oledData

stretchedColumns

get value at

3

stretchedColumns

get value at

3

stretchedColumns

get value at

4

call

oledData

stretchedColumns

get value at

4

stretchedColumns

get value at

5

stretchedColumns

get value at

5

+

function

oledWriteText

row col text

if

oledPresent

then

set

smallText

to

text

if

length of smallText

>

21

or

col

then

set

smallText

to

substring of smallText from 0 of length 21 or col

+

call

oledSetPosition

row col * 6

for

charIndex

from

0

to

length of smallText - 1

do

set

charCode

to

char code from smallText at charIndex

call

oledData

call

fontByte

charCode

1

call

fontByte

charCode

1

call

fontByte

charCode

2

call

oledData

call

fontByte

charCode

3

call

fontByte

charCode

4

0

+

function

oledWriteRow

row text

set

rowText

to

text

while

length of rowText

<

21

do

set

rowText

to

join

rowText

call

oledWriteText

row 0 rowText

call

oledWriteText

row 1 rowText

call

oledWriteText

row 2 rowText

call

oledWriteText

row 3 rowText

call

oledWriteText

row 4 rowText

call

oledWriteText

row 5 rowText

call

oledWriteText

row 6 rowText

call

oledWriteText

row 7 rowText

oledClear A step of the program
Blanks the whole screen.
Used by: deviceTurnsOff, setupOled - **Uses:** oledData, oledSetPosition

function

oledWriteBig

row col text

if

oledPresent

then

set

bigText

to

text

if

length of bigText

>

10

then

set

bigText

to

substring of bigText from 0 of length 10

+

for

half

from

0

to

1

do

call

oledSetPosition

row ++ half col * 6

for

bigCharIndex

from

0

to

length of bigText - 1

do

set

bigCharCode

to

char code from bigText at bigCharIndex

for

columnIndex

from

0

to

5

do

if

columnIndex

<

5

then

set

glyphColumn

to

call

fontByte

bigCharCode columnIndex

+

if

half

=

0

then

stretchedColumns

add value

nibbleStretch

get value at

remainder of glyphColumn / 16

to end

else

stretchedColumns

add value

nibbleStretch

get value at

floor

glyphColumn / 16

to end

+

call

oledData

stretchedColumns

get value at

0

stretchedColumns

get value at

1

stretchedColumns

get value at

1

call

oledData

stretchedColumns

get value at

1

stretchedColumns

get value at

2

stretchedColumns

get value at

2

call

oledData

stretchedColumns

get value at

3

stretchedColumns

get value at

3

stretchedColumns

get value at

4

call

oledData

stretchedColumns

get value at

4

stretchedColumns

get value at

5

stretchedColumns

get value at

5

+

function

oledWriteText

row col text

if

oledPresent

then

set

smallText

to

text

if

length of smallText

>

21

or

col

then

set

smallText

to

substring of smallText from 0 of length 21 or col

+

call

oledSetPosition

row col * 6

for

charIndex

from

0

to

length of smallText - 1

do

set

charCode

to

char code from smallText at charIndex

call

oledData

call

fontByte

charCode

1

call

fontByte

charCode

1

call

fontByte

charCode

2

call

oledData

call

fontByte

charCode

3

call

fontByte

charCode

4

0

+

function

oledWriteRow

row text

set

rowText

to

text

while

length of rowText

<

21

do

set

rowText

to

join

rowText

call

oledWriteText

row 0 rowText

call

oledWriteText

row 1 rowText

call

oledWriteText

row 2 rowText

call

oledWriteText

row 3 rowText

call

oledWriteText

row 4 rowText

call

oledWriteText

row 5 rowText

call

oledWriteText

row 6 rowText

call

oledWriteText

row 7 rowText

oledWriteBig A step of the program - needs: row, col, text
Writes double-size text, two rows tall and up to 10 characters. It is used for the big UV number.
Used by: showUVReading - **Uses:** fontByte, oledData, oledSetPosition

function

oledInitCommands

call

oledCommand

213

call

oledCommand

128

call

oledCommand

166

call

oledCommand

63

call

oledCommand

211

call

oledCommand

9

call

oledCommand

64

if

oledType

=

2

then

call

oledCommand

173

call

oledCommand

135

else

call

oledCommand

141

call

oledCommand

28

call

oledCommand

32

call

oledCommand

2

+

call

oledCommand

161

call

oledCommand

269

call

oledCommand

218

call

oledCommand

18

call

oledCommand

129

call

oledCommand

207

call

oledCommand

217

call

oledCommand

241

call

oledCommand

219

call

oledCommand

64

call

oledCommand

164

call

oledCommand

166

call

oledCommand

175

+

function

oledInitCommands

call

oledCommand

213

call

oledCommand

128

call

oledCommand

166

call

oledCommand

63

call

oledCommand

211

call

oledCommand

9

call

oledCommand

64

if

oledType

=

2

then

call

oledCommand

173

call

oledCommand

135

else

call

oledCommand

141

call

oledCommand

28

call

oledCommand

32

call

oledCommand

2

+

call

oledCommand

161

call

oledCommand

269

call

oledCommand

218

call

oledCommand

18

call

oledCommand

129

call

oledCommand

207

call

oledCommand

217

call

oledCommand

241

call

oledCommand

219

call

oledCommand

64

call

oledCommand

164

call

oledCommand

166

call

oledCommand

175

+

oledInitCommands A step of the program
The standard list of instructions that wakes up a 128 by 64 dot screen.
Used by: setupOled - **Uses:** oledCommand

function

loadFont

set

hex

to

join

hex

to

000000000000005f00000007000700147f147f1424247f2a...

set

hex

to

join

hex

to

21c0014083e08140803e0800950300000000000000000...

set

hex

to

join

hex

to

2141454083181412771827454545393c4e49493901719905...

set

hex

to

join

hex

to

24100141414141408412214080201510986324979413e7e1...

set

hex

to

join

hex

to

7f090909013e4149497a7f0808007f08417f41002040413f...

set

hex

to

join

hex

to

1413e7f090909003e4151213e7f091929464e49494931010...

set

hex

to

join

hex

to

07097000070151494543007f4141002040010200041417f...

set

hex

to

join

hex

to

84438384444442038444487f3854545418007e0901020c5...

set

hex

to

join

hex

to

00417f40007c041804787c08040478384444444307c141414...

set

hex

to

join

hex

to

6207c1c28402013c4020402c4420102040c5950503c445...

set

hexDigits

to

0123456789abccde

set

fontBytes

to

empty array

for

fontIndex

from

0

to

474

do

set

highDigit

to

hexDigits

find index of char from hex at fontIndex * 2

set

lowDigit

to

hexDigits

find index of char from hex at fontIndex * 2 * 1

fontBytes

add value

highDigit * 16 ++ lowDigit

to end

+

function

loadFont

set

hex

to

join

hex

to

000000000000005f00000007000700147f147f1424247f2a...

set

hex

to

join

hex

to

21c0014083e08140803e0800950300000000000000000...

set

hex

to

join

hex

to

2141454083181412771827454545393c4e49493901719905...

set

hex

to

join

hex

to

24100141414141408412214080201510986324979413e7e1...

set

hex

to

join

hex

to

7f090909013e4149497a7f0808007f08417f41002040413f...

set

hex

to

join

hex

to

1413e7f090909003e4151213e7f091929464e49494931010...

set

hex

to

join

hex

to

07097000070151494543007f4141002040010200041417f...

set

hex

to

join

hex

to

84438384444442038444487f3854545418007e0901020c5...

set

hex

to

join

hex

to

00417f40007c041804787c08040478384444444307c141414...

set

hex

to

join

hex

to

6207c1c28402013c4020402c4420102040c5950503c445...

set

hexDigits

to

0123456789abccde

set

fontBytes

to

empty array

for

fontIndex

from

0

to

474

do

set

highDigit

to

hexDigits

find index of char from hex at fontIndex * 2

set

lowDigit

to

hexDigits

find index of char from hex at fontIndex * 2 * 1

fontBytes

add value

highDigit * 16 ++ lowDigit

to end

+

loadFont A step of the program
The shape of every letter, number and symbol, 5 dots wide by 7 tall, stored as text and unpacked when the device starts.
Used by: setupOled

function

oledWriteText

row col text

if

oledPresent

then

set

smallText

to

text

if

length of smallText

>

21

or

col

then

set

smallText

to

substring of smallText from 0 of length 21 or col

+

call

oledSetPosition

row col * 6

for

charIndex

from

0

to

length of smallText - 1

do

set

charCode

to

char code from smallText at charIndex

call

oledData

call

fontByte

charCode

1

call

fontByte

charCode

1

call

fontByte

charCode

2

call

oledData

call

fontByte

charCode

3

call

fontByte

charCode

4

0

+

function

oledWriteText

row col text

if

oledPresent

then

set

smallText

to

text

if

length of smallText

>

21

or

col

then

set

smallText

to

substring of smallText from 0 of length 21 or col

+

call

oledSetPosition

row col * 6

for

charIndex

from

0

to

length of smallText - 1

do

set

charCode

to

char code from smallText at charIndex

call

oledData

call

fontByte

charCode

1

call

fontByte

charCode

1

call

fontByte

charCode

2

call

oledData